

Software Transactional Memory (STM)

Diogo Gomes de Araújo (202510267)

May 29, 2026

Contents

1	Introduction	1
2	Concurrency and Locks	1
3	Transactional Memory (TM)	3
4	STM	3
4.1	Monads, IO and Concurrent Haskell	4
4.2	STM Monad	4
5	A Transactional Order Book	4
6	Conclusion	6

1 Introduction

This report was written as an assignment for the Advanced Topics in Functional Programming course at Faculdade de Ciências da Universidade do Porto. It explores Software Transactional Memory (STM) for modeling concurrent programs in purely functional programming languages, like Haskell, and presents a case study on a stock exchange's order book. The source code is publicly available in the following repository.

2 Concurrency and Locks

Throughout many years, the growth of computational power within a single processing unit was enough to overcome execution-time limitations, but as the advances in computer hardware decelerate we must think of software-level optimizations to make better use of the machines we have at hand [8]. Modern hardware provides an interface for executing tasks in parallel using a common set of resources. Though more efficient, systems must now formalize concurrency models for simultaneous resource access in non-deterministic order.

Implementing such mechanisms requires hardware support for atomic instructions, namely "compare-and-swap", "test-and-set", and "fetch-and-add". These atomic operations guarantee that shared memory can be accessed and modified safely without interference from concurrent tasks. They form the basis for implementing synchronization primitives such as locks, semaphores, and lock-free data structures, ensuring correct execution and preventing race conditions in multithreaded and multicore systems.

Mutexes or locks are widely adopted by programmers as they define a simple model for isolating critical sections of execution, where data consistency is crucial, by limiting access to shared assets to a single running task using a lock-based system. While these structures provide a necessary boundary for data integrity, their over-reliance on mutual exclusion often imposes a rigid sequential bottleneck that suppresses the potential

parallelism of modern multicore architectures. Furthermore, the management of these primitives introduces risks such as deadlocks, priority inversion, and the non-trivial overhead of kernel-level thread suspension, which can severely degrade performance in systems requiring high responsiveness. Consequently, developers must carefully balance the simplicity of lock-based synchronization against the potential for decreased throughput when scaling to large numbers of concurrent threads.

In *Beautiful Concurrency* [3], Simon Peyton Jones warns about the underlying complexity of simplistic concurrency constructs with a case study over bank account transfers. A similar challenge arises when handling a stock exchange’s order book. Recent *Nasdaq* market summaries show the extraordinary scale at which modern exchanges operate, processing around 70 million orders daily [5].

Metric	Share Volume	Dollar Volume
Total Volume	10,073,462,044	\$527,866,082,028
Block Volume	2,399,217,446	–

Additional Statistics	Value
Number of Issues	5,622
Number of MPs	605
Total Trades	69,442,435
Block Trades	77,017

Table 1: *Nasdaq* Daily Market Summary in May 19, 2026

The vast majority of these transactions are not initiated by human traders but by proprietary algorithms executing decisions in microseconds, with no manual intervention. As computational infrastructure continues to expand and access to low-latency processing becomes increasingly democratized, the volume and velocity of machine-generated orders is expected to grow further. This trajectory places mounting pressure on the software systems that underpin these exchanges, making the design of scalable, robust abstractions not merely an engineering convenience but an operational necessity.

For pedagogical purposes consider a trimmed down example, where a stock’s order book is defined by two separate priority queues (for bids and asks), ordered by price and time of arrival. An unwise approach for modeling concurrent access in an imperative programming language, such as Rust [9], using *locks* as the primary synchronization primitive would be:

```

1 struct OrderBook<Order> {
2     bid_orders: Arc<Mutex<PriorityQueue<Order>>>,
3     ask_orders: Arc<Mutex<PriorityQueue<Order>>>,
4 }

```

Let there be two threads, A and B, executing in parallel. Thread A acquires the lock for `bid_orders`, adds an order, and then attempts to acquire the lock for `ask_orders` to remove the lowest ask from the queue while still holding the `bid_orders` lock. Thread B follows a similar pattern, first acquiring the `ask_orders` lock and adding an ask, and then attempting to acquire the `bid_orders` lock to remove the lowest bid while still holding the `ask_orders` lock.

```

1 if let Ok(mut bid_orders) = order_book.bid_orders.lock() {
2     bid_orders.insert(order);
3
4     let lowest_ask = order_book.ask_orders.lock()?.remove();
5     ...
6 }

```

This naive implementation can lead to a deadlock, since A may be waiting to acquire `ask_orders` while B is waiting to acquire `bid_orders`, with each lock already held by the other thread. As a result, neither thread can proceed and both remain blocked indefinitely.

Additionally, even in a scenario where A and B release their locks before entering the second critical section, as the number of threads competing for lock acquisition increases, concurrent transaction execution becomes highly inefficient. Only one entity may execute a critical section while holding the lock, blocking all others.

3 Transactional Memory (TM)

Nir Shavit defines a transaction as *a finite sequence of local and shared memory machine instructions* [7] executed in an atomic way. In transactional memory, database operations are handled through sequences of `read_transactional` and `write_transactional` instructions that compose a transaction. Each transaction either terminates successfully, when all operations occur, updating the shared memory state, or aborts and is re-queued for later execution when one or more fail to run. Though simple, this model allows for implementing non-blocking and lock-free concurrent access to shared data structures.

Earlier transactional memory implementations required hardware-specific synchronization mechanisms to create lock-free shared memory locations on multiprocessor systems [2]. A transactional cache would store writes temporarily until a `commit` or `abort` instruction was called. This hardware-centric approach effectively decoupled the logical progression of transactions from the immediate contention of shared memory, allowing multiple threads to operate in parallel while maintaining the illusion of atomicity through speculative execution. By buffering modifications, these systems could ensure that partial results remained invisible to other processors until a consistent state was confirmed, thereby simplifying the design of complex data structures that would otherwise require fine-grained, error-prone locking strategies.

A major drawback was the limited amount of memory in the transactional cache. If a transaction's dataset exceeded the size of the cache, it would unavoidably lead to abortion. This capacity constraint depended on a rigid limit on the number of shared memory locations a single atomic block could touch, effectively preventing the use of hardware-based transactional memory for large-scale data manipulations. Consequently, the reliance on fixed-size buffers meant that even theoretically correct algorithms would suffer from inevitable starvation in the presence of memory-intensive operations, undermining the flexibility and scalability that such architectural primitives were initially intended to provide.

4 STM

Software Transactional Memory (STM) expands upon this framework by providing a software-based abstraction for managing concurrent access to shared memory without relying on hardware support [7]. Contrary to locks, STM is optimistic as it allows threads to modify shared memory without regard for others, logging every read and write operation.

In STM, a thread executes a transaction in isolation. If no other thread has written to the same memory addresses during execution, the changes are committed. Otherwise, the transaction is aborted, all changes are discarded, and it is re-queued to run again. The reader is tasked with re-evaluating data consistency after executing an atomic instruction sequence, unlike *mutex*'s writer responsibility, that may be carelessly overlooked. By treating memory operations as atomic transactions rather than protected critical sections, STM mitigates the risks of priority inversion and deadlock inherent in lock-based systems. It allows the system to prioritize progress over exclusivity, as threads that do not conflict can continue their work in parallel, only resolving disputes at the final commit phase. This shift allows for more predictable scaling, as the system handles contention internally, ensuring that the shared state remains consistent even when subjected to the high-frequency demands of complex, parallel workloads.

In the context of functional programming, composability arises as an inherent trait of a transactional abstraction [1]. Since transactions encapsulate effects within an ephemeral log, they behave as first-class values that can be sequenced, combined, and passed as arguments before any effect is committed to shared state.

4.1 Monads, IO and Concurrent Haskell

Monads are an algebraic abstraction for sequencing computations within a context. In Haskell, a monad is any type constructor satisfying the `Monad` typeclass, which requires a `return` function to lift a pure value into the context and a bind operator (`>>=`) to chain instructions.

The `IO` monad [6] bridges pure computation with effectful interaction by wrapping the return type of a function with `IO`, signaling the lazy-evaluation compiler and the programmer, preventing implicitly passing effects from obfuscating the operational semantics of the program. For instance, `putStr` and `getLine` have types:

```
1 putStr  :: String -> IO ()
2 getLine :: IO String
```

Effectful operations can be composed using the monadic bind (`>>=`) operator or a semantically equivalent syntactic sugar called *do* notation:

```
1 main :: IO ()
2
3 — bind
4 main = getLine >>= putStr
5
6 — do notation
7 main = do
8     input <- getLine
9     putStr input
```

Concurrency primitives are encoded in `Control.Concurrent` [4] using monadic `IO` properties to enforce strict manipulation of side effects, ensuring that operations such as reading shared state or spawning threads remain predictable:

```
1 forkIO :: IO a -> IO ThreadId
```

4.2 STM Monad

Transactions, in Haskell, are composed using the `STM` monad, provided by `Control.Concurrent.STM`. A transactional variable of type `TVar` can be atomically mutated and read through sequences of `readTVar` and `writeTVar` operations:

```
1 readTVar :: TVar a -> STM a
2 writeTVar :: TVar a -> a -> STM ()
```

As such, transactions are composed into pure `STM` values and only executed when lifted into `IO`, using the `atomically` operator defined by the following type:

```
1 atomically :: STM a -> IO a
```

This mechanism ensures that all transactional actions are executed atomically, meaning that intermediate states are never observable by other threads. If a transaction cannot be completed due to conflicting updates, it is re-evaluated, ensuring that only consistent results are produced. This allows developers to compose complex concurrent operations declaratively, relying on `STM` to preserve consistency and isolation across concurrent interactions.

5 A Transactional Order Book

The design of a stock exchange order book is a natural application of `STM`, as it involves frequent concurrent modifications to shared state under strict consistency requirements. This section presents a purely functional implementation of a transactional order book, in Haskell, where all state-changing operations are expressed

within the STM monad, ensuring they remain composable, atomic, and isolated. Like before, the order book consists of two priority queues, one for buy orders and one for sell orders, together with a transactional clock incremented upon the arrival of each incoming order:

```

1 data TOrderBook c = TOrderBook { bidOrders :: TPriorityQueue (Order c),
2                                   askOrders :: TPriorityQueue (Order c),
3                                   clock      :: TVar (Timestamp) }

```

Unlike a lock-based design, no explicit synchronisation primitives guard this state. Instead, the runtime detects conflicts dynamically and retries computations as needed, without programmer intervention.

Each order pairs a header with a timestamp. The header encodes whether the order is a buy or sell request and its associated limit price:

```

1 data OrderType = Buy | Sell
2   deriving (Show, Eq)
3
4 data Header c = Header { limitAmount :: c,
5                           orderType   :: OrderType
6 } deriving (Show, Eq)
7
8 data Order c = Order { header      :: Header c,
9                       timestamp    :: Timestamp
10 } deriving (Show, Eq)

```

We derive an `Ord` instance to ensure that better-priced orders are prioritized, with ties resolved in FIFO order through timestamps:

```

1 instance Ord c => Ord (Order c) where
2   compare ordA ordB =
3     compare (limitAmount $ header ordA, timestamp ordA)
4       (limitAmount $ header ordB, timestamp ordB)

```

To preserve global ordering consistency, we assign each order a timestamp atomically inside the STM context:

```

1 stamp :: Ord c => TOrderBook c -> Header c -> STM (Order c)
2 stamp ordBook h = do
3   ts <- readTVar $ clock ordBook
4   _   <- writeTVar (clock ordBook) (ts + 1)
5   return $ newOrder h ts

```

Initialization creates empty transactional priority queues alongside a fresh clock, composing naturally within the STM monad:

```

1 newTOrderBook :: Ord c => STM (TOrderBook c)
2 newTOrderBook = do
3   bidOrds <- newTPriorityQueue
4   askOrds <- newTPriorityQueue
5   clk     <- newTVar 0
6   return $ TOrderBook { clock      = clk,
7                           bidOrders = bidOrds,
8                           askOrders = askOrds }

```

The insertion and removal operations remain concise because synchronisation concerns are delegated entirely to the STM runtime. New orders are first timestamped atomically and then inserted into the corresponding transactional priority queue according to their type. Since both timestamp assignment and queue insertion occur within the same transaction, the operation preserves consistency even under concurrent submissions.

Order removal follows a similarly simple structure. Depending on the requested side of the book, the operation atomically extracts the highest-priority element from the corresponding queue. Because the queues

themselves are transactional data structures, concurrent insertions and removals compose safely without requiring explicit locking or manual conflict resolution:

```

1  addTOrderBook :: Ord c => Header c -> TOrderBook c -> STM ()
2  addTOrderBook hd ordBook = do
3      ord <- stamp ordBook hd
4      case orderType hd of
5          Buy  -> insertTPriorityQueue ord (bidOrders ordBook)
6          Sell -> insertTPriorityQueue ord (askOrders ordBook)
7
8  removeTOrderBook :: Ord c => OrderType -> TOrderBook c -> STM (Maybe (Order c))
9  removeTOrderBook t ordBook = case t of
10     Buy  -> pollTPriorityQueue $ bidOrders ordBook
11     Sell -> pollTPriorityQueue $ askOrders ordBook

```

6 Conclusion

In synthesis, STM offers a principled and composable alternative to lock-based concurrency, addressing many of the pitfalls that make concurrent programming notoriously error-prone. By treating shared-memory access as atomic transactions rather than guarded critical sections, STM eliminates the risks of deadlock and priority inversion while enabling a degree of parallelism that locking fundamentally suppresses.

The stock exchange order book case study illustrates these advantages concretely. Operations that would require careful lock ordering and extensive reasoning about thread interleaving in an imperative setting reduce, under STM, to straightforward monadic compositions. The runtime assumes responsibility for conflict detection and retry logic, freeing the programmer to express intent rather than synchronization strategy.

Haskell’s type system reinforces this discipline further. The separation between the STM and IO monads makes transactional boundaries explicit and statically enforced, preventing transactional code from leaking effects and ensuring that atomicity is never accidental. Composability, often treated as a secondary concern in concurrent systems, emerges as a first-class property. Transactions can be sequenced, combined, and passed as values before any effect reaches shared state.

References

- [1] Tim Harris, Simon Marlow, Simon Peyton-Jones, and Maurice Herlihy. Composable memory transactions. In *Proceedings of the tenth ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 48–60, 2005.
- [2] Maurice Herlihy and J Eliot B Moss. Transactional memory: Architectural support for lock-free data structures. In *Proceedings of the 20th annual international symposium on Computer architecture*, pages 289–300, 1993.
- [3] Simon Peyton Jones. Beautiful concurrency. *Beautiful Code: Leading Programmers Explain How They Think*, pages 385–406, 2007.
- [4] Simon Peyton Jones, Andrew Gordon, and Sigbjorn Finne. Concurrent haskell. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT symposium on Principles of programming languages-POPL’96*, 1996.
- [5] Nasdaq Trader. Daily market summary. <https://www.nasdaqtrader.com/Trader.aspx?id=DailyMarketSummary>, 2026. Accessed: 2026-05-24.
- [6] Bryan O’Sullivan, John Goerzen, and Donald Bruce Stewart. *Real world haskell: Code you can believe in.* ” O’Reilly Media, Inc.”, 2008.

- [7] Nir Shavit and Dan Touitou. Software transactional memory. In *Proceedings of the fourteenth annual ACM symposium on Principles of distributed computing*, pages 204–213, 1995.
- [8] Herb Sutter et al. The free lunch is over: A fundamental turn toward concurrency in software. *Dr. Dobbs's journal*, 30(3):202–210, 2005.
- [9] The Rust Project Developers. `std::sync::mutex`. <https://doc.rust-lang.org/std/sync/struct.Mutex.html>, 2026. Accessed: 2026-05-26.